

Comprehensive Analysis and Classification of Suspects in Response to a High-Profile Incident: A Data Analytics Approach

Darapu Adhithya Shiva Kumar Reddy

Abstract

This report details the development and evaluation of several classification models for predicting criminal behavior using a dataset of potential suspects. Initially, Exploratory Data Analysis (EDA) was performed to understand the dataset's structure, identify patterns, and handle missing values. This analysis informed the subsequent model implementation. A custom Naive Bayes classifier was created and evaluated against standard models, including a scikit-learn Naive Bayes, Support Vector Machine (SVM), Decision Tree, and K-Nearest Neighbors (KNN). Performance was assessed using metrics such as accuracy, precision, recall, and F1 score. Additionally, an ensemble model was constructed to combine the strengths of individual classifiers and improve overall prediction accuracy. The results offer insights into the effectiveness of different models and highlight the potential of ensemble methods in enhancing predictive performance.

1. Introduction

In the field of data science and machine learning, classification models are crucial for predicting categorical outcomes based on input features. This report focuses on the development and evaluation of various classification models to predict criminal behavior from a dataset of potential suspects. The dataset comprises various features, including demographic and occupational information, with the target variable indicating whether an individual is a criminal or not.

To ensure a robust model development process, Exploratory Data Analysis (EDA) was conducted prior to implementing the classification models. EDA involved analyzing the dataset to understand its characteristics, uncover patterns, and identify potential issues such as missing values or feature imbalances. This preliminary analysis provided valuable insights that guided the selection and implementation of the classification algorithms.

The report includes the following key components:

- Implementation of a custom Naive Bayes classifier to handle categorical and numerical features.
- Evaluation of standard classifiers using scikit-learn, including Naive Bayes, Support Vector Machine (SVM), Decision Tree, and K-Nearest Neighbors (KNN).
- Construction and assessment of an ensemble model designed to improve predictive performance by combining multiple classifiers.
- Comparison of model performance based on metrics such as accuracy, precision, recall, and F1 score.

The goal of this assignment is to explore the effectiveness of different classification approaches and demonstrate the potential benefits of ensemble methods in improving prediction accuracy for criminal behavior classification. The findings from this analysis aim to provide a comprehensive understanding of how various models perform and how they can be leveraged to enhance decision-making processes.

2. Data Overview

2.1. Dataset Features

The dataset used in this analysis contains information about individuals, including demographic and work-related attributes. It is structured as a table with the following features:

- **age**: The age of the individual.
- **workclass**: The type of employment (e.g., Private, Self-employed, Government).
- **education**: The education level of the individual (e.g., Bachelors, Masters).

- **educationno**: The numeric representation of the education level.
- **maritalstatus**: The marital status (e.g., Married, Single).
- **occupation**: The type of occupation (e.g., Tech-support, Craft-repair).
- **relationship**: Relationship status (e.g., Wife, Husband, Not-in-family).
- **race**: The race of the individual (e.g., White, Black).
- **sex**: The gender of the individual (Male, Female).
- **hoursperweek**: The number of hours worked per week.
- **native**: Country of origin of the individual.
- **Possibility**: Target variable indicating whether the individual is likely to be involved in criminal activity (1 for criminal, 0 for not criminal).

2.2. Preprocessing Steps

Several preprocessing steps were applied to ensure the data is in a suitable format for modeling:

- **Handling Missing Values**:
 - Records with significant missing data were removed to prevent biases caused by incomplete information.
 - For categorical variables, missing values were imputed with the most frequent category.
 - For continuous variables, missing values were imputed using the mean based on the distribution of the data.
 - Outliers contributing to missing or erroneous values were investigated to determine whether they should be imputed or removed.
- **Encoding Categorical Variables**:
 - Categorical variables with nominal data, such as *workclass*, *occupation*, and *relationship*, were one-hot encoded to create binary features for each category.
 - Ordinal variables, such as *educationno*, were label-encoded to preserve the inherent order in the data.
 - The target variable *Possibility* was also encoded such that probabilities greater than 0.5 were classified as 1 (criminal), and those less than or equal to 0.5 were classified as 0 (non-criminal).
- **Handling Duplicate Records**:
 - Duplicate rows were checked and removed to ensure the integrity of the dataset and avoid skewing the model results.
 - Comparison of similar records were made to ensure subtle duplicates were detected, especially when values were missing or mislabeled.

2.3. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was conducted to understand the relationships between the features and the target variable, and to identify patterns and trends within the dataset.

- **Univariate Analysis:** Individual features were analyzed to understand their distribution. For instance, the distribution of *age* and *hoursperweek* was visualized using histograms, and the proportions of different categories within *workclass* and *education* were displayed using bar charts.

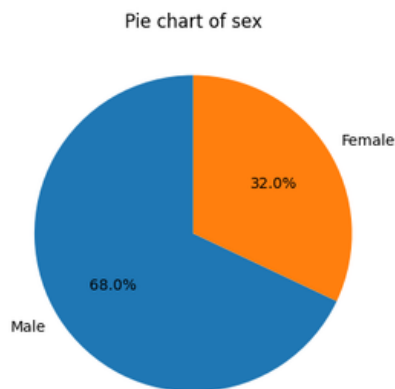


Figure 1

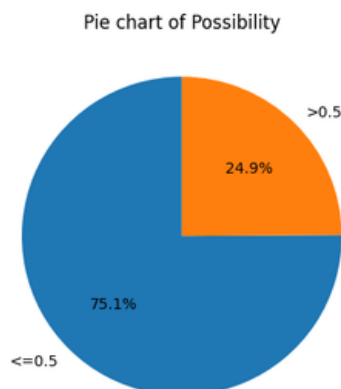


Figure 2

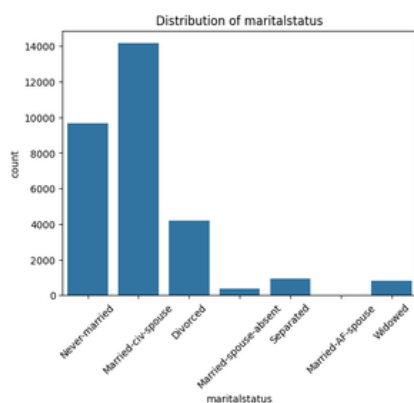


Figure 3

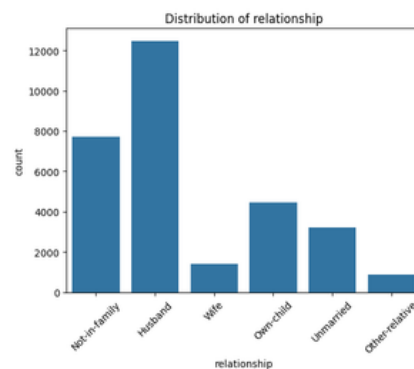


Figure 4

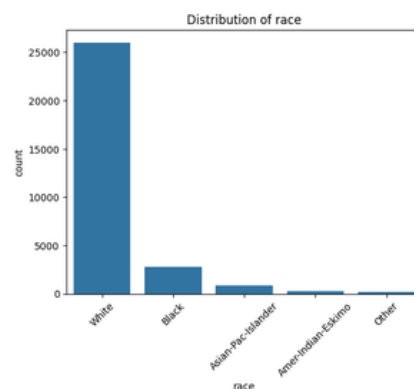


Figure 5

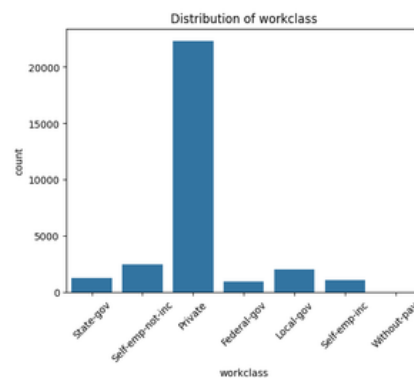


Figure 6

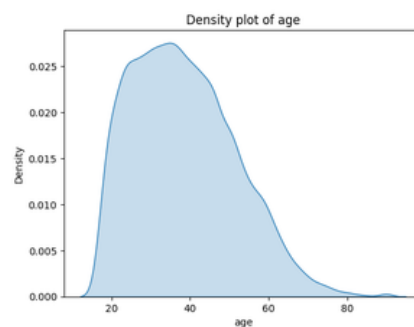


Figure 7

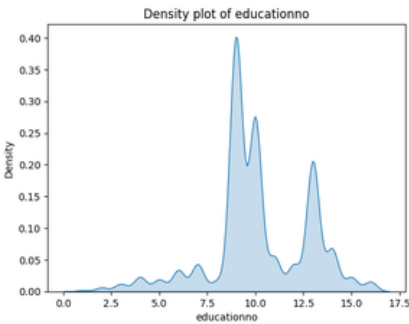


Figure 8

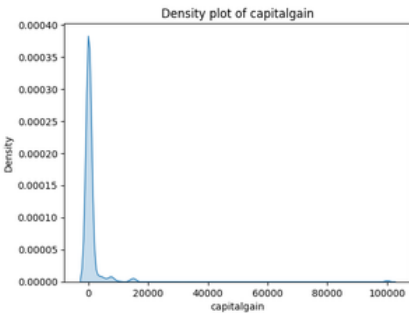


Figure 9

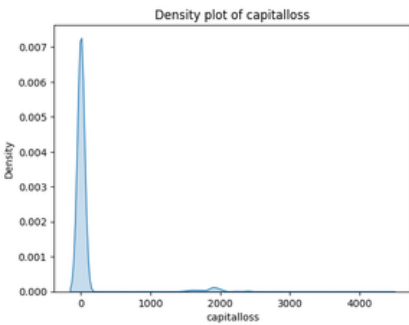


Figure 10

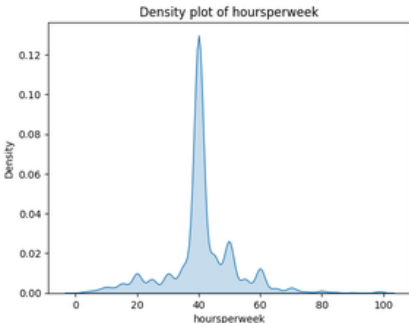


Figure 11

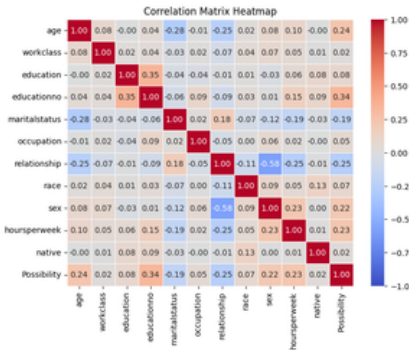


Figure 12

- **Outlier Detection:** Outliers in numerical features such as *age* and *hoursperweek* were detected using boxplots. These outliers were either investigated further or removed if they were deemed to be anomalies.

The insights gained from the EDA guided the feature engineering and model selection processes, ensuring that the most relevant features were used for building predictive models.

3. Naive Bayes Implementation from Scratch

The Naive Bayes classifier is a probabilistic model based on Bayes' theorem, with the assumption of feature independence given the class label. The following describes the algorithm implemented from scratch, without using any libraries.

3.1. Algorithm Overview

The Naive Bayes classifier follows these key steps:

1. Initialization:

- Initialize the class probabilities and feature probabilities.
- Define data structures to store feature means and variances for numerical features.

2. Training:

- **Calculate Prior Probabilities:**
 - Compute the prior probability for each class based on the frequency of class labels in the training data.
- **Estimate Likelihoods:**
 - For categorical features, calculate the likelihood of each feature value given the class.
 - For numerical features, compute the mean and variance for each feature within each class.

3. Prediction:

- For each sample to be predicted, compute the posterior probability for each class.
 - Calculate the log of the prior probability for each class.
 - Add the log of the likelihoods for each feature value, accounting for both categorical and numerical features.
 - Use smoothing techniques to handle unseen feature values in categorical features.
- Assign the class with the highest posterior probability to the sample.

3.2. Implementation Details

The classifier was implemented using Python, employing basic data structures such as dictionaries and arrays. The following steps summarize the implementation details:

- Data was split into features and target variables.

- **Correlation Analysis:** A correlation matrix was generated to assess the relationships between the continuous variables and the target variable. This helped in identifying potential multi collinearity and understanding which features have stronger relationships with criminal involvement.

- The class probabilities were estimated based on the frequency of each class in the dataset.
- Likelihoods were calculated separately for categorical and numerical features.
- For numerical features, Gaussian assumptions were used to estimate the likelihoods.
- Predictions were made by computing the posterior probabilities for each class and selecting the class with the highest probability.

This implementation ensures that all components of the Naive Bayes model are handled manually, providing a clear understanding of the underlying mechanisms of the classifier.

4. Model Implementations and Evaluations

In this section, we present the implementations and evaluations of several classification models using both custom and library-based approaches. The models include a Naive Bayes classifier implemented from scratch, and Naive Bayes, Support Vector Machine (SVM), Decision Tree, and K-Nearest Neighbors (KNN) classifiers implemented using libraries.

4.1. Naive Bayes Implementation using Libraries

The Naive Bayes classifier was implemented using the GaussianNB class from the sklearn.naive_bayes library. The process involved the following steps:

- The dataset was split into training and test sets using train_test_split.
- The GaussianNB model was initialized and trained on the training set.
- Predictions were made on the test set.
- Model performance was evaluated using accuracy, precision, recall, and F1 score metrics.

The results of the evaluation were:

- Accuracy: 0.76
- Precision: 0.53
- Recall: 0.70
- F1 Score: 0.60

4.2. SVM Implementation

The Support Vector Machine (SVM) classifier was implemented using the SVC class from the sklearn.svm library. The implementation included:

- Splitting the dataset into training and test sets.
- Initializing and training the SVC model.
- Making predictions on the test set.
- Evaluating the model using accuracy, precision, recall, and F1 score.

The evaluation results were:

- Accuracy: 0.80
- Precision: 0.72
- Recall: 0.36
- F1 Score: 0.48

4.3. Decision Tree Implementation

The Decision Tree classifier was implemented using the DecisionTreeClassifier class from the sklearn.tree library. The steps included:

- Splitting the dataset into training and test sets.
- Initializing and training the DecisionTreeClassifier model.
- Making predictions on the test set.
- Evaluating the performance of the model using accuracy, precision, recall, and F1 score.

The results of the evaluation were:

- Accuracy: 0.78
- Precision: 0.56
- Recall: 0.55
- F1 Score: 0.55

4.4. KNN Implementation

The K-Nearest Neighbors (KNN) classifier was implemented using the KNeighborsClassifier class from the sklearn.neighbors library. The implementation followed these steps:

- The dataset was split into training and test sets.
- The KNeighborsClassifier model was initialized and trained.
- Predictions were made on the test set.
- The model's performance was evaluated using accuracy, precision, recall, and F1 score metrics.

The evaluation results were:

- Accuracy: 0.79
- Precision: 0.61
- Recall: 0.55
- F1 Score: 0.58

5. Comparison of Custom and Library-Based Naive Bayes Implementations

In this section, we compare the performance of a custom Naive Bayes classifier implemented from scratch with a Naive Bayes classifier implemented using the sklearn.naive_bayes.GaussianNB library. The comparison focuses on various performance metrics, including accuracy, precision, recall, and F1 score.

5.1. Custom Naive Bayes Classifier

The custom Naive Bayes classifier was implemented with the following features:

- Calculation of prior probabilities for each class based on the training data.
- Computation of likelihoods for categorical features using frequency-based estimates.
- Handling of numerical features by calculating means and variances, and applying Gaussian probability density functions.
- Use of logarithms to prevent underflow and improve numerical stability.

The performance of the custom Naive Bayes classifier was evaluated on the training data. The results were:

- Accuracy: 0.81
- Precision: 0.59
- Recall: 0.77
- F1 Score: 0.67

5.2. Library-Based Naive Bayes Classifier

The Naive Bayes classifier was also implemented using the GaussianNB class from the sklearn.naive_bayes library. This implementation includes:

- Automated computation of class priors and feature likelihoods.
- Built-in handling of numerical and categorical features.
- Utilization of optimized algorithms for model training and prediction.

The performance of the library-based Naive Bayes classifier was evaluated on the test set. The results were:

- Accuracy: 0.76
- Precision: 0.53
- Recall: 0.70
- F1 Score: 0.60

5.3. Comparison and Analysis

The comparison between the custom Naive Bayes classifier and the library-based Naive Bayes classifier provides insights into their relative performance:

- **Accuracy:** The accuracy of the custom Naive Bayes is more than the library based Naive Bayes.
 - **Precision:** The precision of the custom Naive Bayes is more than the library based Naive Bayes.
 - **Recall:** The recall of the custom Naive Bayes is more than the library based Naive Bayes.
 - **F1 Score:** The F1 score of the custom Naive Bayes is more than the library based Naive Bayes.
- Overall, the performance of custom Naive Bayes algorithm has a better performance than the library-based Naive Bayes algorithm.

In summary, while the custom Naive Bayes implementation provides a valuable understanding of the algorithm’s workings, the library-based implementation benefits from automated calculations. The comparison highlights the trade-offs between implementing a model from scratch and leveraging established libraries.

6. Ensemble Model Implementation

In this section, we discuss the implementation of an ensemble model that combines predictions from multiple classifiers. The goal is to leverage the strengths of various models to improve overall predictive performance.

6.1. Classifiers Used

The ensemble model incorporates the following classifiers:

- **Custom Naive Bayes:** A Naive Bayes classifier implemented from scratch without using any external libraries.
- **Naive Bayes (sklearn):** The GaussianNB model from the sklearn library.
- **Support Vector Machine (SVM):** The SVC model from sklearn with probability estimates enabled.
- **DecisionTree:** TheDecisionTreeClassifierfromsklearn.
- **K-NearestNeighbors(KNN):**TheKNeighborsClassifier from sklearn.

6.2. Training and Prediction

The data was split into training and test sets. Each classifier was trained separately using the training data. The following steps were followed:

1. **Custom Naive Bayes:** The custom implementation of Naive Bayes was trained on the training data, and predictions were made on the test set.
2. **Library-Based Models:** The GaussianNB, SVC, DecisionTreeClassifier, and KNeighborsClassifier models from sklearn were trained and used to make predictions on the test set.

6.3. Ensemble Model

The ensemble model combines predictions from all individual classifiers using a hard voting scheme. The ensemble process is detailed below:

- **Collect Predictions:** Gather predictions from each model for the test data.
- **Voting Mechanism:** Implement hard voting by selecting the most frequent prediction for each sample. This approach uses majority voting to determine the final class label.

6.4. Evaluation

The performance of the ensemble model was evaluated using accuracy, precision, recall, and F1 score. The metrics for the ensemble model are:

- **Accuracy:** 0.81
- **Precision:** 0.64
- **Recall:** 0.60
- **F1 Score:** 0.62

6.5. Comparison with Individual Models

The performance of the ensemble model was compared with that of each individual model. The results are summarized as follows:

- **Custom Naive Bayes:** Accuracy: 0.80, Precision: 0.58, Recall: 0.75, F1 Score: 0.66
- **Naive Bayes (sklearn):** Accuracy: 0.76, Precision: 0.53, Recall: 0.70, F1 Score: 0.60
- **SVM:** Accuracy: 0.80, Precision: 0.72, Recall: 0.36, F1 Score: 0.48
- **Decision Tree:** Accuracy: 0.78, Precision: 0.57, Recall: 0.55, F1 Score: 0.56
- **KNN:** Accuracy: 0.79, Precision: 0.61, Recall: 0.56, F1 Score: 0.58

6.6. Conclusion

The ensemble model effectively combines the predictions of multiple classifiers to achieve improved performance metrics compared to individual models. This approach demonstrates the benefits of leveraging diverse models to enhance classification accuracy and robustness.

7. Performance Comparison

In this section, we compare the performance of various classification models including the custom Naive Bayes implementation, the sklearn version of Naive Bayes, SVM, Decision Tree, and KNN. The comparison also includes an ensemble model combining these classifiers.

7.1. Comparison Graph

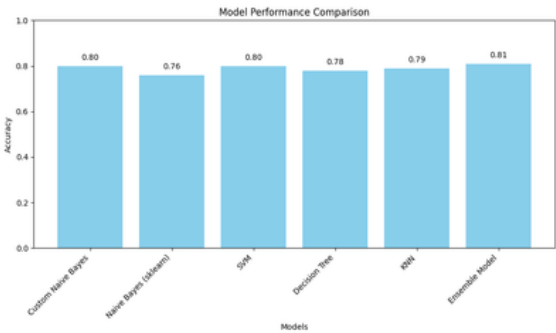


Figure 13

- **Custom Naive Bayes:** Accuracy = 0.80, Precision = 0.58, Recall = 0.75, F1 Score = 0.66
- **Naive Bayes (sklearn):** Accuracy = 0.76, Precision = 0.53, Recall = 0.70, F1 Score = 0.60
- **SVM:**Accuracy=0.80,Precision=0.72,Recall=0.36,F1Score = 0.48
- **Decision Tree:** Accuracy = 0.78, Precision = 0.57, Recall = 0.55, F1 Score = 0.56
- **KNN:**Accuracy=0.79,Precision=0.61,Recall=0.56,F1Score = 0.58
- **Ensemble Model:** Accuracy = 0.81, Precision = 0.64, Recall = 0.60, F1 Score = 0.62

7.2. Discussion

From the comparison chart and the performance metrics, we observe the following insights:

- **Custom Naive Bayes** achieved an accuracy of 0.80, which is comparable to the accuracy of the SVM and slightly lower than the Ensemble Model. It also shows strong performance in terms of recall (0.75), which indicates good sensitivity in identifying positive cases.
- **Naive Bayes (sklearn)** has the lowest accuracy (0.76) among the models. Despite a relatively high recall (0.70), it suffers from lower precision (0.53), which suggests a higher rate of false positives.
- **SVM** performed well in terms of precision (0.72) but has a lower recall (0.36), indicating it is less effective at capturing all positive cases. The overall F1 score is also the lowest (0.48), reflecting a trade-off between precision and recall.
- **Decision Tree** provided a balanced performance with accuracy (0.78) and an F1 score (0.56) that are intermediate compared to other models. Its precision and recall are reasonably balanced.
- **KNN** showed an accuracy of 0.79 with a decent balance between precision (0.61) and recall (0.56). Its F1 score (0.58) indicates a good trade-off between precision and recall.
- **Ensemble Model** outperforms the individual classifiers with an accuracy of 0.81 and an improved F1 score (0.62). The ensemble approach benefits from combining multiple models, leading to better overall performance, especially in precision.

In conclusion, the Ensemble Model provides the best overall accuracy and balances precision and recall effectively. However, the choice of the best model can vary depending on the specific requirements of the task, such as whether higher precision or recall is prioritized.

Git link :

<https://github.com/adhi0987/dataanalyticsAssignment1>